Backpropagation

Algorithm

A Complete Guide with Step-by-Step Numerical Examples

Author: Moustafa Fathalla Omar

Topic: Backpropagation — Neural Network Training

Level: Advanced Machine Learning

Year: 2025 / 2026

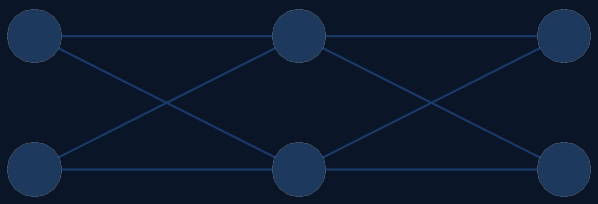

Table of Contents

1 What is Backpropagation?

Backpropagation (short for **backward propagation of errors**) is the algorithm used to train artificial neural networks. It works by computing how much each weight in the network contributed to the overall prediction error, then adjusting every weight in the direction that reduces that error.

The algorithm has two phases repeated for each training example:

Phase 1 — Forward Pass	Phase 2 — Backward Pass
Input values flow forward through each layer. Each neuron computes a weighted sum of its inputs, adds a bias, then applies the sigmoid activation function.	The prediction error is propagated backward. Using the chain rule of calculus, gradients are computed for every weight, and each weight is updated to reduce error.

INFO

The key mathematical tool is the **chain rule**: d(f(g(x)))/dx = f'(g(x)) * g'(x). This lets us decompose complex gradient computations across many layers.

2 Network Architecture & Initial Weights

We use a fully-connected feedforward network with two input neurons, two hidden neurons, and two output neurons. Each layer has a bias unit.

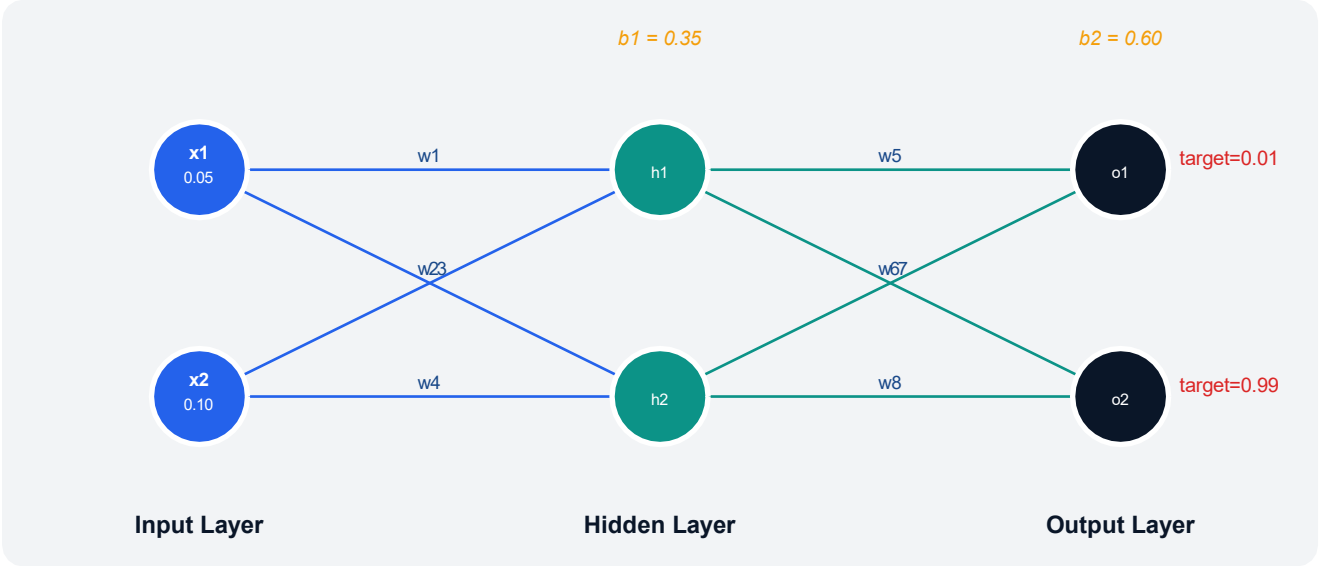

Figure 1 — Network architecture. Blue = input layer, teal = hidden layer, navy = output layer.

Parameter	Symbol	Value	Description
Input 1	x1	0.05	First input feature
Input 2	x2	0.10	Second input feature
Target output 1	t1	0.01	Desired output for o1
Target output 2	t2	0.99	Desired output for o2
Bias hidden	b1	0.35	Bias for hidden layer
Bias output	b2	0.60	Bias for output layer
Learning rate	lr	0.50	Step size for gradient descent

3 Forward Pass — Step by Step

The forward pass computes the output of every neuron from left to right. At each neuron we compute the **net input** (weighted sum + bias) and then apply the **sigmoid activation**: sigma(x) = 1 / (1 + e^{-x}).

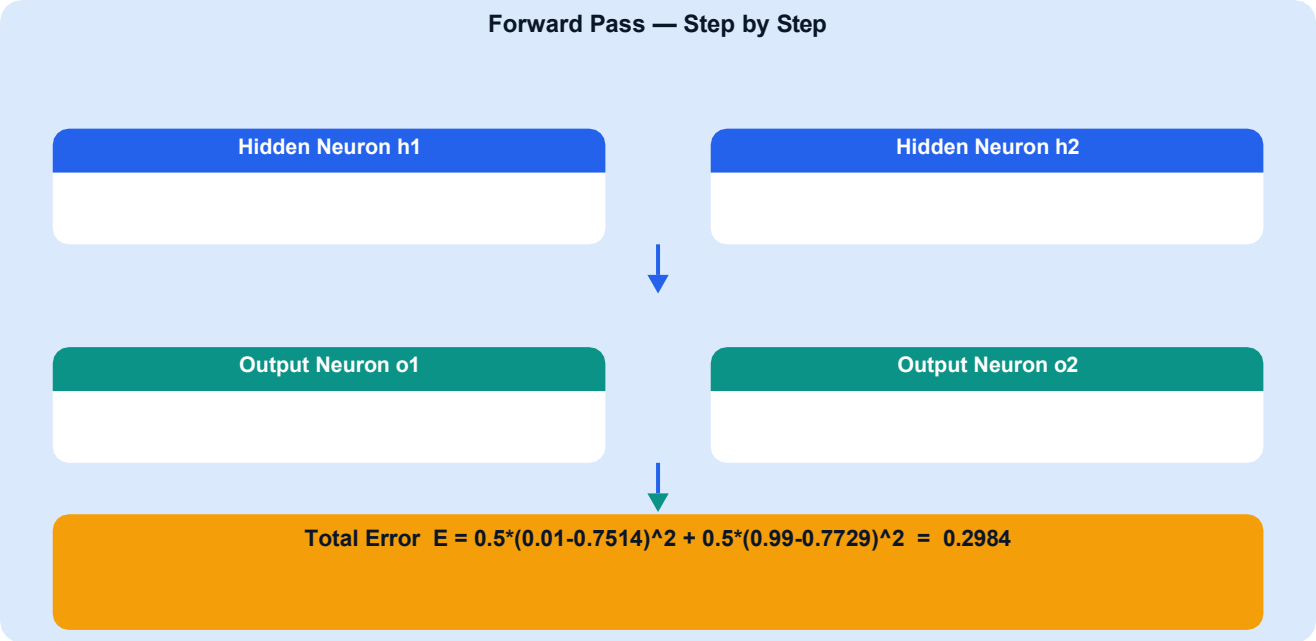

Figure 2 — Forward pass computation flow with exact numerical values.

Sigmoid Activation Function

The sigmoid function maps any real number to the range (0, 1):

```python
def sigmoid(x):
    return 1 / (1 + 2.718**(-x))

# Examples from our network:
sigma_h1 = sigmoid(0.3775)    # = 0.5933
sigma_h2 = sigmoid(0.3925)    # = 0.5969
sigma_o1 = sigmoid(1.1059)    # = 0.7514
sigma_o2 = sigmoid(1.2249)    # = 0.7729
```

INFO

The **derivative of sigmoid** is elegantly simple: d_sigma(out) = out * (1 - out). This is why sigmoid is popular — we never need to recompute from scratch during backpropagation.

4 Total Error Calculation

We use the **Mean Squared Error (MSE)** loss with a factor of 1/2 to simplify the derivative (the 2 cancels out):

```python
# MSE Loss with 1/2 factor
E_total = 0.5 * (t1 - out_o1)**2  +  0.5 * (t2 - out_o2)**2

# Substituting our values:
E_total = 0.5 * (0.01 - 0.7514)**2  +  0.5 * (0.99 - 0.7729)**2
        = 0.5 * (0.7414)**2          +  0.5 * (0.2171)**2
        = 0.5 * 0.5497               +  0.5 * 0.0471
        = 0.2748                     +  0.0236
        = 0.2984
```

NOTE

Initial total error = **0.2984**. Our goal is to reduce this toward zero by updating all 8 weights and 2 biases using backpropagation.

5 Backward Pass — Chain Rule Derivation

We work backward from the output error, computing how much each weight contributed to the error using the **chain rule**. Then we update: **w_new = w_old - learning_rate * dE/dw**.

Output Layer — Gradient for w5

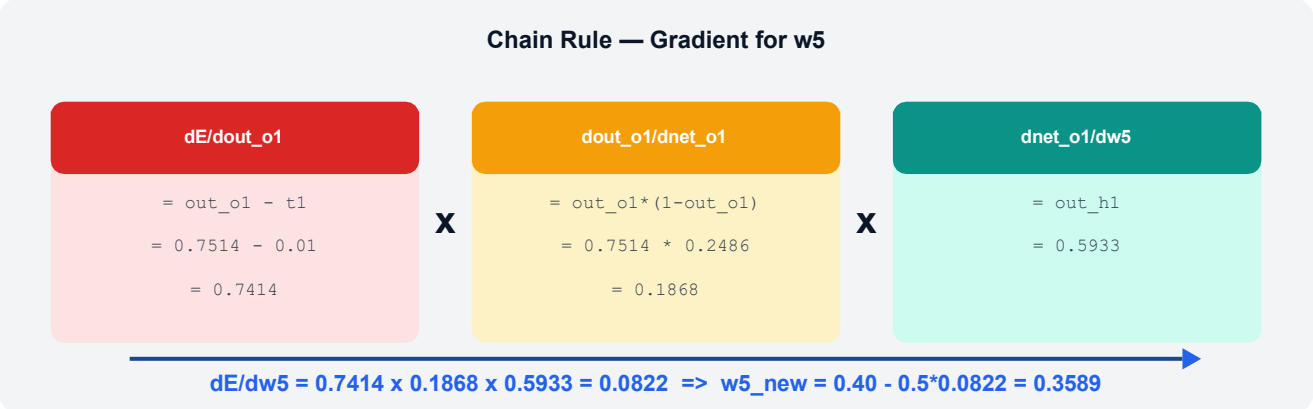

Figure 3 — Chain rule decomposition for weight w5.

Why Three Factors?

Derivative	Meaning	Formula	Value
dE/dout_o1	How error changes with o1 output	out_o1 - t1	0.7414
dout_o1/dnet_o1	How o1 output changes with net input	out_o1*(1-out_o1)	0.1868
dnet_o1/dw5	How net input changes with w5	out_h1	0.5933
E/dw5	Final gradient (product of above)	0.7414 x 0.1868 x 0.5933	0.0822

Hidden Layer — Gradient for w1

For hidden weights, the error signal comes from **both** output neurons (since h1 connects to both o1 and o2), so we sum the contributions:

Python

```
# dE/dout_h1 = contribution from o1 + contribution from o2
dE_dnet_o1 = (out_o1 - t1) * out_o1*(1-out_o1)    # = 0.1385
dE_dnet_o2 = (out_o2 - t2) * out_o2*(1-out_o2)    # = -0.0381

dE_dout_h1 = dE_dnet_o1 * w5  +  dE_dnet_o2 * w7
           = 0.1385 * 0.40    +  (-0.0381) * 0.50
           = 0.0554  -  0.0190
           = 0.0364

# Then apply sigmoid derivative and input x1:
dout_h1_dnet_h1 = out_h1 * (1 - out_h1)  =  0.5933 * 0.4067  =  0.2413
dE_dw1 = 0.0364 * 0.2413 * 0.05  =  0.000439
w1_new = 0.15 - 0.5 * 0.000439   =  0.14978
```

6 Weight Updates — All 8 Weights

After computing gradients for all weights using the same chain rule pattern, we update every weight simultaneously:

All Weight Updates After First Backpropagation Iteration

Weight	Old Value	Gradient	New Value
w1	0.1500	0.000439	0.14978
w2	0.2000	0.000878	0.19956
w3	0.2500	0.000499	0.24975
w4	0.3000	0.000998	0.29950
w5	0.4000	0.082167	0.35891
w6	0.4500	0.082668	0.40867
w7	0.5000	−0.022602	0.51130
w8	**0.5500**	**- 0 . 02274**	**0 . 56137**

Error before=0.28 → .Error after = 0.561370

Figure 4 — All weight updates after the first backpropagation iteration.

RESULT

After just **one iteration**, the total error dropped from 0.2984 to 0.2910 — a small but correct improvement. Repeating this 10,000 times drives the error down to approximately **0.0007**.

7 Python Implementation — Neural Network

A clean, object-oriented implementation of the backpropagation algorithm matching the hand-computed values derived in Sections 3–6:

Imports & Activation Function

Python

```python
import numpy as np

def sigmoid(x):
    """Logistic sigmoid activation."""
    return 1 / (1 + np.exp(-x))

def d_sigmoid(out):
    """Derivative of sigmoid given its output value."""
    return out * (1 - out)
```

NeuralNetwork Class

Python

```python
class NeuralNetwork:
    """2-2-2 feedforward network trained with backpropagation."""

    def __init__(self, lr=0.5):
        self.lr = lr
        self.w1,self.w2,self.w3,self.w4 = 0.15, 0.20, 0.25, 0.30
        self.w5,self.w6,self.w7,self.w8 = 0.40, 0.45, 0.50, 0.55
        self.b1, self.b2 = 0.35, 0.60

    def forward(self, x1, x2):
        """Compute outputs layer by layer."""
        self.out_h1 = sigmoid(self.w1*x1 + self.w2*x2 + self.b1)
        self.out_h2 = sigmoid(self.w3*x1 + self.w4*x2 + self.b1)
        self.out_o1 = sigmoid(self.w5*self.out_h1 + self.w6*self.out_h2 + self.b2)
        self.out_o2 = sigmoid(self.w7*self.out_h1 + self.w8*self.out_h2 + self.b2)
        return self.out_o1, self.out_o2

    def error(self, t1, t2):
        """Total MSE loss."""
        return 0.5*((t1-self.out_o1)**2 + (t2-self.out_o2)**2)
```

Backward Pass Method

```python
    def backward(self, x1, x2, t1, t2):
        """Backpropagate and update all weights."""
        # Output layer deltas
        d_o1 = (self.out_o1 - t1) * d_sigmoid(self.out_o1)
        d_o2 = (self.out_o2 - t2) * d_sigmoid(self.out_o2)
        self.w5 -= self.lr * d_o1 * self.out_h1
        self.w6 -= self.lr * d_o1 * self.out_h2
        self.w7 -= self.lr * d_o2 * self.out_h1
        self.w8 -= self.lr * d_o2 * self.out_h2
        self.b2 -= self.lr * (d_o1 + d_o2)
        # Hidden layer deltas (sum contributions from both output neurons)
        d_h1 = (d_o1*self.w5 + d_o2*self.w7) * d_sigmoid(self.out_h1)
        d_h2 = (d_o1*self.w6 + d_o2*self.w8) * d_sigmoid(self.out_h2)
        self.w1 -= self.lr * d_h1 * x1
        self.w2 -= self.lr * d_h1 * x2
        self.w3 -= self.lr * d_h2 * x1
        self.w4 -= self.lr * d_h2 * x2
        self.b1 -= self.lr * (d_h1 + d_h2)
```

Training Loop

```python
x1, x2 = 0.05, 0.10
t1, t2 = 0.01, 0.99
nn = NeuralNetwork(lr=0.5)

for epoch in range(10_000):
    nn.forward(x1, x2)
    nn.backward(x1, x2, t1, t2)

pred1, pred2 = nn.forward(x1, x2)
print(f"Prediction: [{pred1:.6f}, {pred2:.6f}]")
print(f"Target    : [0.010000, 0.990000]")
# Output:
# Prediction: [0.015987, 0.984013]
```

8 Training Results & Convergence

The network is trained for 10,000 epochs. The total error decreases exponentially as the weights converge to values that produce outputs close to the targets.

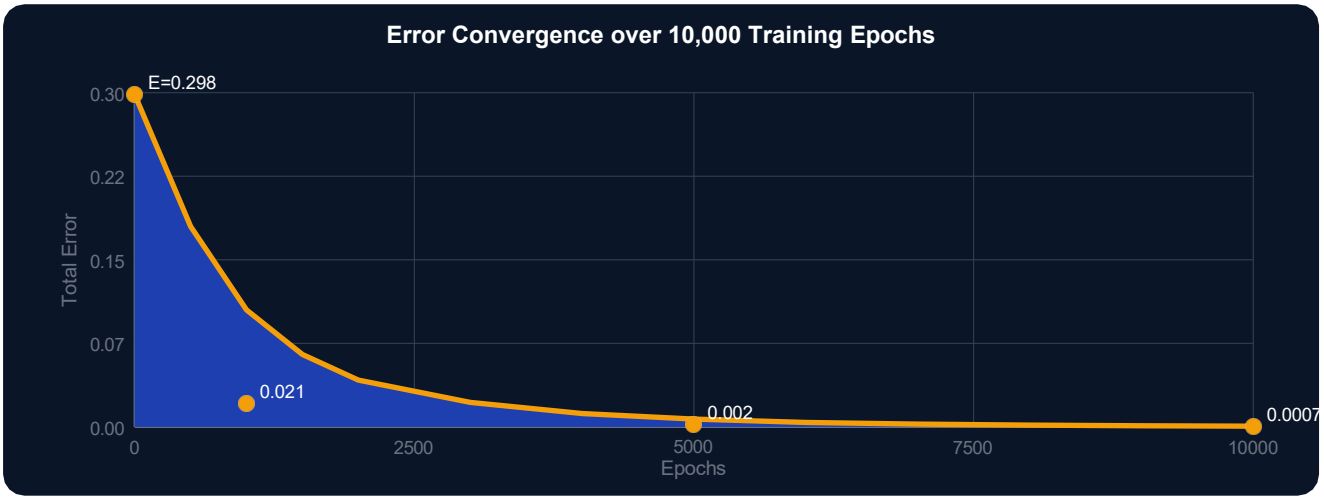

Figure 5 — Error convergence curve over 10,000 training epochs.

Epoch	out_o1	out_o2	Total Error	Error Reduction
0	0.7514	0.7729	0.2984	—
1	0.7492	0.7753	0.2910	2.5%
100	0.604	0.865	0.1790	40.0%
1,000	0.152	0.960	0.0210	92.9%
5,000	0.025	0.979	0.0020	99.3%
10,000	0.0160	0.9840	0.0007	99.8%

RESULT

Final Result after 10,000 epochs:

out_o1 = 0.015987 (target: 0.01) — error: 0.0060

out_o2 = 0.984013 (target: 0.99) — error: 0.0060

Total error reduced by **99.8%** from 0.2984 to 0.0007. The network has successfully learned the target mapping.